

# Computação de Alto Desempenho

Henrique de Oliveira Araújo

n<sup>o</sup> USP: 13863950

Jorge Augusto Salgado Salhani

n<sup>o</sup> USP: 8927418

Matheus Velloso de Andrade

n<sup>o</sup> USP: 12421725

Rafael Guimaraes Borecki de Carvalho

n<sup>o</sup> USP: 14604271

Thiago Aragão da Silva

n<sup>o</sup> USP: 10753062

Setembro, 2026

## 1 Código fonte

O código fonte utilizado como base para a construção deste relatório está disponível no GitHub e pode ser acessado pelo link abaixo:

 Computação de Alto Desempenho (Equipe) : Trabalho 1

Link (direto): <https://github.com/jorgesalhani/HighPerformanceComputingTeam/tree/main/Trabalho1>

## 2 Análise qualitativa

Nesta seção, analisaremos os resultados obtidos de modo qualitativo, avaliando se as alterações dos parâmetros apresentam mudanças esperadas na simulação. Para este fim, manteremos os parâmetros gerais seguintes, conforme entrada padrão presente no arquivo disponibilizado *entrada\_carga\_pequena.txt*

entrada\_carga\_pequena.txt

```
400 500 80 4 2027 35
0 1 3
3 2
100 83
200 250
300 416
15 70 150 130 165
35 245 320 355 335
```

Neste arquivo de entrada podemos organizar as informações conforme as tabelas abaixo.

| Matriz (LxC) | 400x500 |
|--------------|---------|
| Passos (P)   | 80      |
| Seed         | 2027    |
| Limiar       | 35      |

Table 1: configurações principais.

| Focos (F) (L, C) |
|------------------|
| (100, 83)        |
| (200, 250)       |
| (300, 416)       |

Table 2: Focos de incêndio.

| Zonas (Z)                  |
|----------------------------|
| P: (LI, LF),(CI, CF)       |
| 15: [(70,150), (130, 165)] |
| 35: [(245,320), (355,355)] |

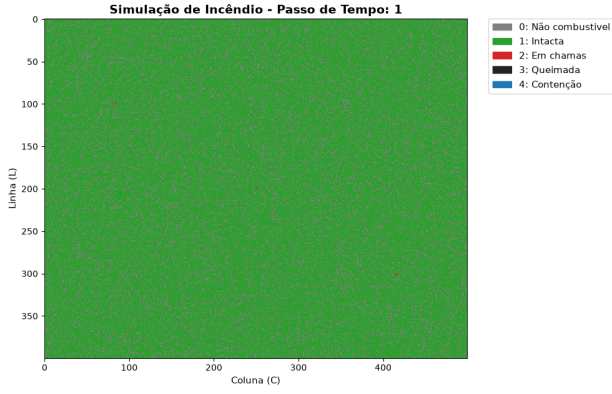
Table 3: Zonas de contenção.

A seguir, detalhamos algumas etapas da simulação para embasar as análises qualitativas realizadas.

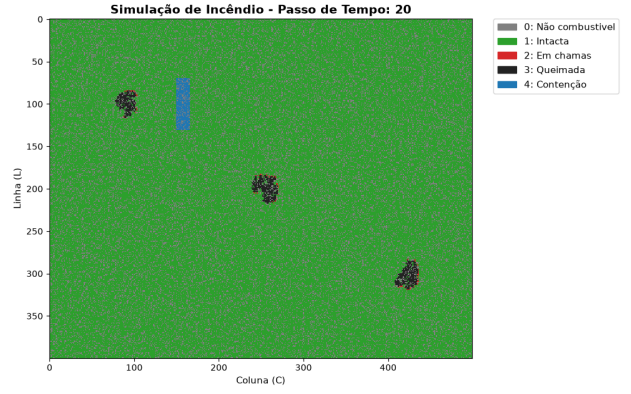
## 2.1 Evolução de estados

Conforme analisado na figura 1, em diferentes momentos temporais em passos  $p$  discretos representamos o mapa de estados para cada célula da matriz  $400 \times 500$ .

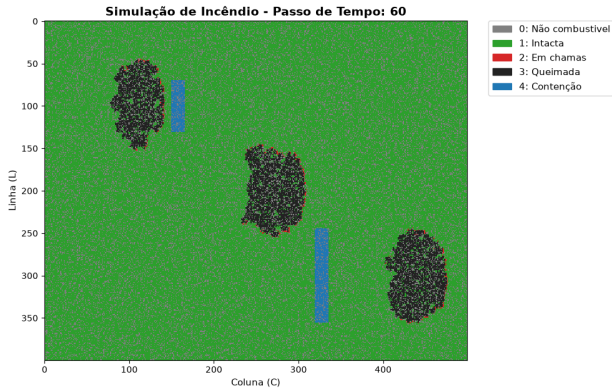
No instante de tempo  $p = 1$ , verificamos apenas três focos de incêndio (pelas configurações iniciais nas coordenadas (100, 83), (200, 250) e (300, 416)). Conforme avançamos para  $p = 20$ , notamos a criação de uma zona de contenção (azul) para a área delimitada pela região [(70, 150), (130, 165)], assim como a evolução de células adjacentes aos focos de incêndio para estados de "queimada" e "em chamas". Para  $p = 60$ , uma nova zona de contenção é estabelecida na região [(245, 320), (355, 355)]. Notamos também uma tendência de dispersão de células queimadas para a direita, conforme direção do vento para leste (conforme configuração de vento (0, 1), intensidade 3, que analisaremos na seção seguinte). Por fim, para  $p = 80$ , já no final da simulação, destacamos a manutenção da zona de contenção, com células queimadas apenas na região não englobada pela zona com início no passo  $p = 15$ .



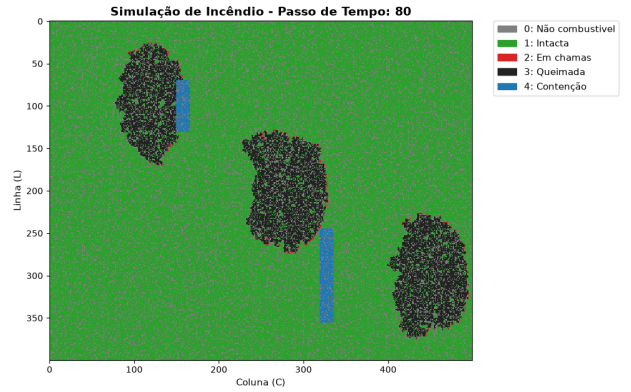
(a) Matriz de estados para  $p = 1$ .



(b) Matriz de estados para  $p = 20$ .



(c) Matriz de estados para  $p = 60$ .



(d) Matriz de estados para  $p = 80$ .

Figure 1: Mapa de estados para tempos crescentes de simulação considerando parâmetros padrão, com configurações de vento (para leste) dadas por: direção (0,1), intensidade 3. Instantes de tempo no passo (a)  $p = 0$  (b)  $p = 20$  (c)  $p = 60$  (d)  $p = 80$ .

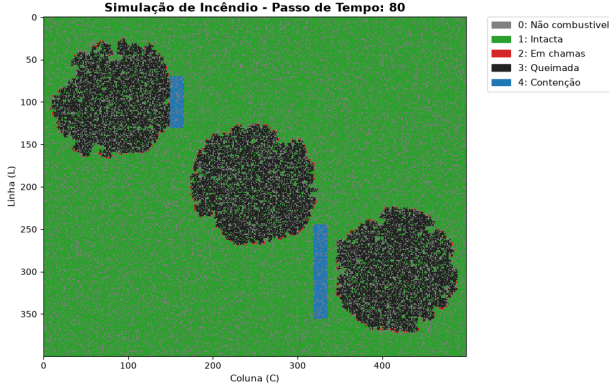
## 2.2 Parâmetros de vento

Neste momento, avaliaremos os efeitos da mudança de parâmetros de vento na simulação.

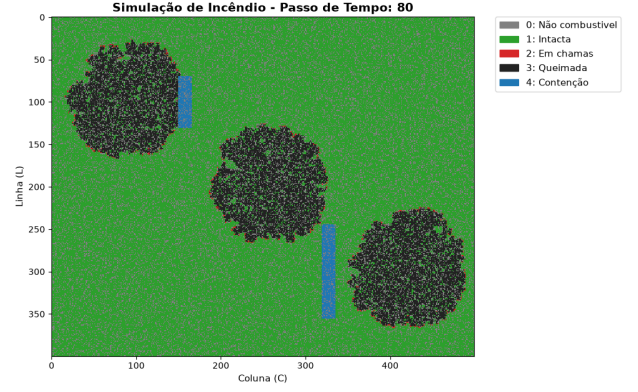
Na figura 1 mantivemos a situação final da simulação ( $p = 80$ ) com a direção original do vento (para leste (0, 1)), mas considerando variações na intensidade do vento. Para intensidade  $V = 0$ , notamos a ausência de viés de propagação das células em chamas e queimadas. O fogo se alastra de modo aproximadamente circular em torno do foco inicial de incêndio. Para  $V = 1$  e  $V = 4$  notamos uma tendência crescente de dispersão das células em chamas e queimadas à direita dos focos de incêndio iniciais, atingindo o limite máximo possível de intensidade, com  $V = 5$ , onde o viés à direita é mais proeminente. As células em chamas e queimadas descreve aproximadamente uma seção de circunferência, com centro nos focos iniciais de incêndio.

Por outro lado, para a figura 3 mantivemos a intensidade padrão de ventos  $V = 3$  e alteramos a direção de vento para (1,-1), sentido sudoeste, para distintos instantes de tempo com  $p = 0$ ,  $p = 20$ ,  $p = 60$  e  $p = 80$ . É visível a

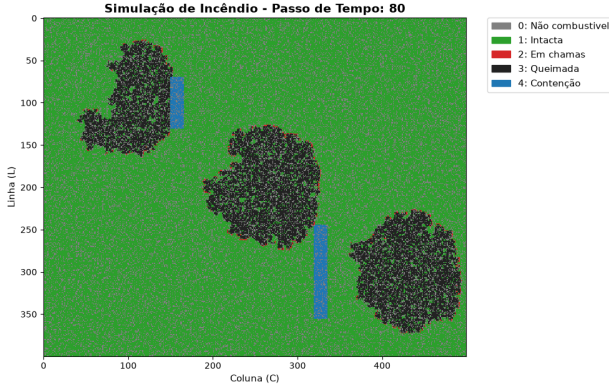
mudança de viés de propagação das células queimadas e em chamas ao longo do tempo, particularmente quando comparadas à figura 1, cujos parâmetros de vento apresentam mesma intensidade mas para leste.



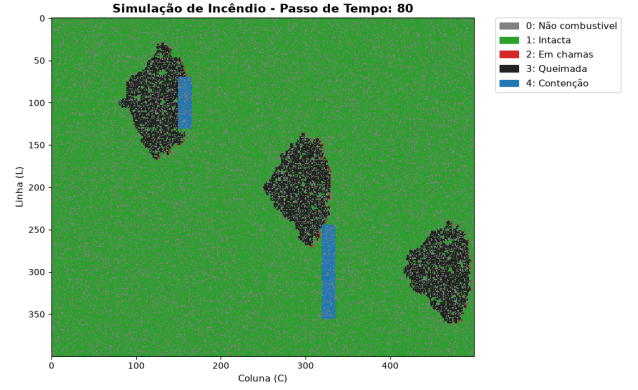
(a) Matriz de estados para intensidade de vento  $V = 0$ .



(b) Matriz de estados para intensidade de vento  $V = 1$ .



(c) Matriz de estados para intensidade de vento  $V = 4$ .



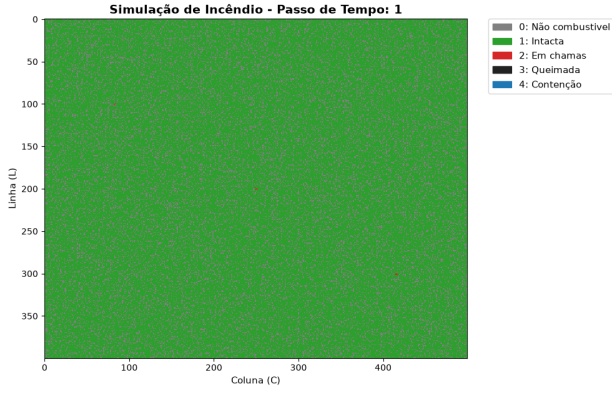
(d) Matriz de estados para intensidade de vento  $V = 5$ .

Figure 2: Mapa de estados para tempos fixos ( $p = 80$ ) de simulação considerando parâmetros padrão, configurações de vento (para leste) com direção (0,1) fixa e intensidades crescentes (a)  $V = 0$  (b)  $V = 1$  (c)  $V = 4$  (d)  $V = 5$ .

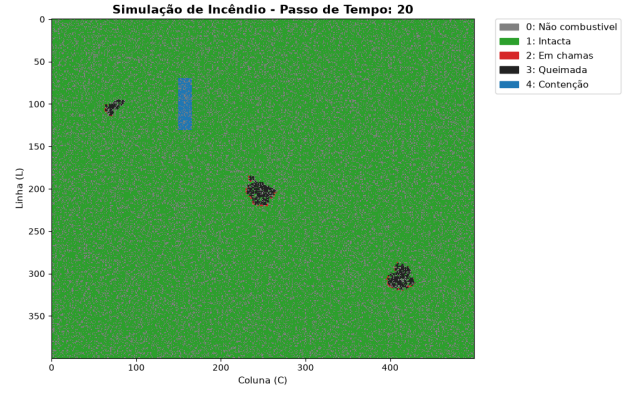
### 2.3 Limiar de incêndio

Analizamos agora as mudanças qualitativas na alteração do limiar de incêndio, ou seja, o valor mínimo para que uma determinada célula entre em ignição e seja alterada do estado "intacta" para "em chamas".

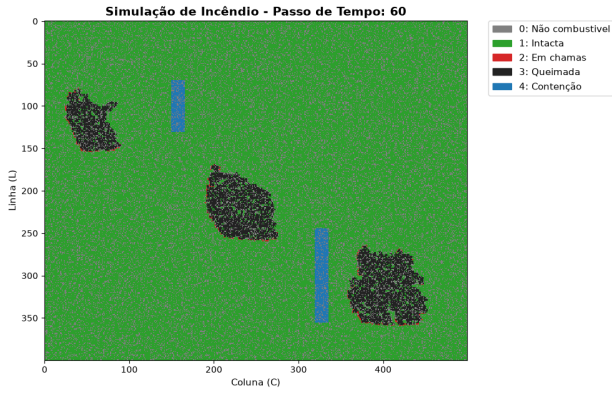
Na figura 4 mantivemos as configurações padrão apresentadas no início do capítulo, alterando o limiar de incêndio com análise do momento final da simulação  $p = 80$ . Para  $L = 3$ , notamos uma dispersão radial em relação aos focos iniciais de incêndio, uma vez que quase todas as células intactas adjacentes a uma célula em chamas serão transferidas para o estado em chamas e, em seguida, para queimadas. Conforme aumentamos o limiar para  $L = 5$  e  $L = 35$ , menor a chance de células intactas sofrerem com as queimadas adjacentes até que, para valores bastante altos  $L = 55$ , algumas poucas células sofrem com as queimadas preexistentes.



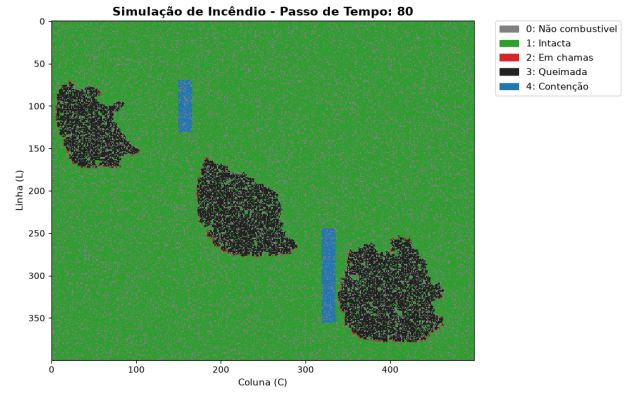
(a) Matriz de estados para o instante  $p = 0$ .



(b) Matriz de estados para o instante  $p = 20$ .



(c) Matriz de estados para o instante  $p = 60$ .



(d) Matriz de estados para o instante  $p = 80$ .

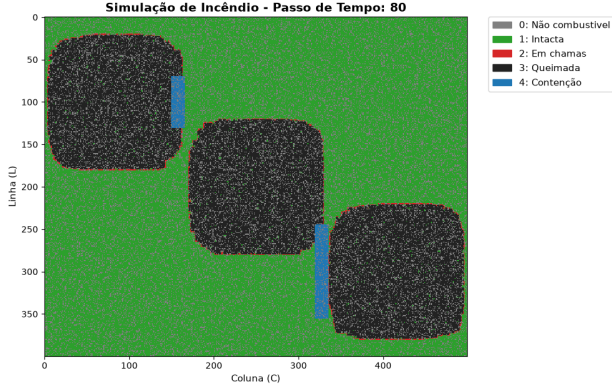
Figure 3: Mapa de estados para tempos crescentes de simulação considerando parâmetros padrão, com configurações de vento (para sudoeste) dadas por: direção (1,-1), intensidade 3. Instantes de tempo no passo (a)  $p = 0$  (b)  $p = 20$  (c)  $p = 60$  (d)  $p = 80$ .

## 2.4 Evolução temporal

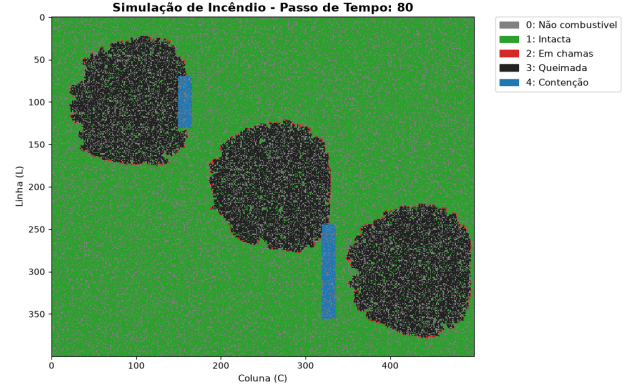
Em última análise, avaliamos a evolução temporal do número de células por estado, mantendo as configurações padrão dadas no início desta seção, com limiar de incêndio iguais a  $L = 10$  e  $L = 45$ .

Na figura 5 Em ambos os cenários conseguimos notar o crescimento contínuo de células queimadas em oposição ao número de células intactas. Também fica evidente nos instantes de tempo  $p = 15$  e  $p = 35$  a aparição de múltiplas células em em estado de contenção.

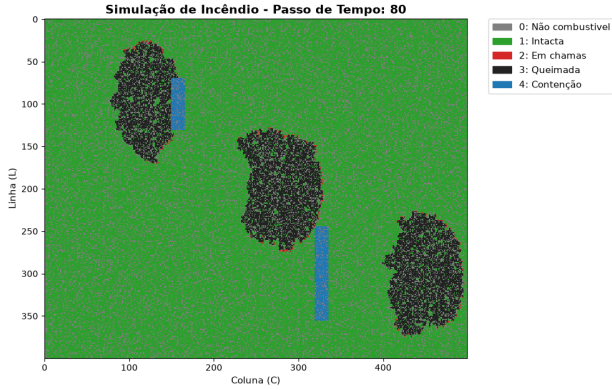
Vale também destacar, quando mantemos o limiar alto de ignição com  $L = 45$ , que o número de queimadas cresce em menor escala em quando comparado ao limiar  $L = 10$ , atingindo um máximo para  $p = 64$ , quando o número de células em chamas passa a reduzir ao longo do tempo.



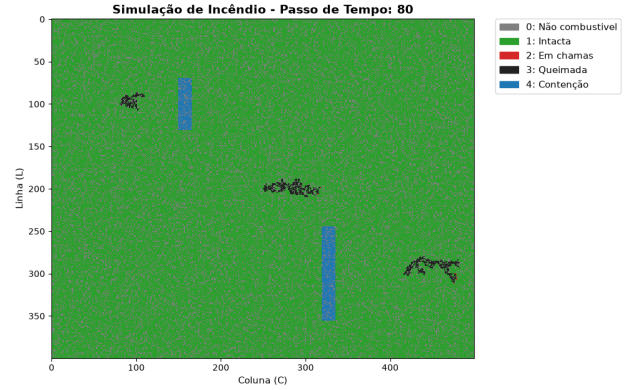
(a) Matriz de estados com limiar de incêndio  $L = 5$ .



(b) Matriz de estados com limiar de incêndio  $L = 25$ .

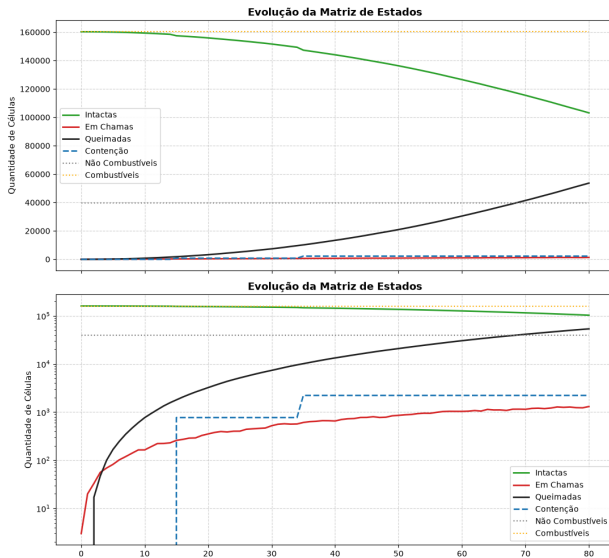


(c) Matriz de estados com limiar de incêndio  $L = 35$ .

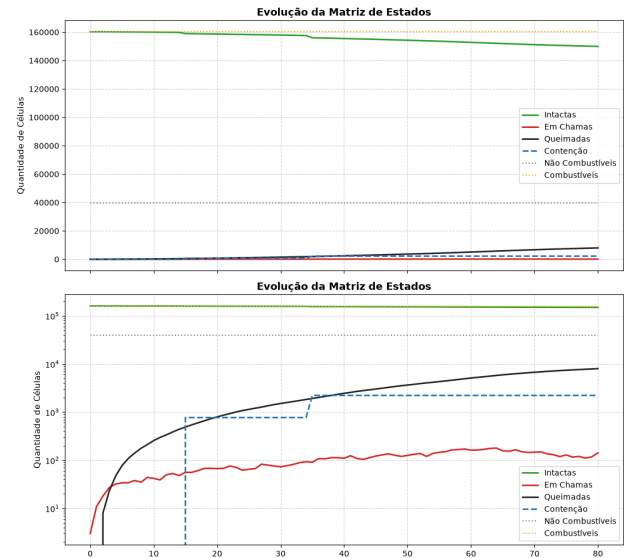


(d) Matriz de estados com limiar de incêndio  $L = 55$ .

Figure 4: Mapa de estados com diversos potenciais mínimos de ocorrência de um novo incêndio (limiar) considerando parâmetros padrão, com configurações de vento fixa (para leste) dadas por: direção (0,1), intensidade 3, e para tempo fixo ( $p = 80$ ), sendo (a) limiar = 5 (b) limiar = 25 (c) limiar = 35 (d)  $p = 55$ .



(a) Evolução temporal do número de estados para limiar de incêndio  $L = 10$ .



(b) Evolução temporal do número de estados para limiar de incêndio  $L = 45$ .

Figure 5: Evolução temporal da matriz de estados considerando os parâmetros padrão, configurações de vento (para leste) com direção (0,1) fixa e (a) limiar = 10 e (b) limiar = 45.

### 3 Análise quantitativa

Uma vez validada a dinâmica dos diferentes parâmetros da simulação, podemos descrever os resultados quantitativos esperados.

Abaixo apresentamos os arquivos de entrada e suas respectivas saídas para a solução sequencial.

**entrada\_carga\_pequena.txt**

```
400 500 80 4 2027 35
0 1 3
3 2
100 83
200 250
300 416
15 70 150 130 165
35 245 320 355 335
```

**output.out**

```
passos: 80
nao_combustiveis: 39704
intactas: 138809
em_chamas: 376
queimadas: 18889
contencao: 2222
total_ignicoes: 372
pico_ignicoes: 66 398
percentual_queimado: 12.02
percentual_protegido: 1.39
checksum: 35988890671224108
tempo: 0.648286
```

#### entrada\_carga\_media.txt

```
1200 1500 100 8 3031 35
1 1 2
6 4
171 125
342 375
514 625
685 876
857 1125
1028 1375
20 250 450 450 470
40 500 780 780 800
60 760 1080 1030 1100
75 950 1280 1150 1300
```

#### output.out

```
passos: 100
nao_combustiveis: 360809
intactas: 1356445
em_chamas: 1191
queimadas: 65510
contencao: 16045
total_ignicoes: 1185
pico_ignicoes: 90 1228
percentual_queimado: 4.63
percentual_protegido: 1.11
checksum: 2450667959220978121
tempo: 10.998731
```

#### entrada\_carga\_grande.txt

```
2500 2500 100 8 4099 35
-1 1 4
10 6
227 125
454 375
681 626
909 875
1136 1125
1363 1375
1590 1626
1818 1875
2045 2125
2272 2375
15 150 300 500 325
30 550 700 900 730
45 950 1100 1350 1135
60 1350 1500 1750 1540
75 1750 1900 2150 1945
90 2050 2200 2400 2250
```

#### output.out

```
passos: 100
nao_combustiveis: 1251181
intactas: 4855039
em_chamas: 1419
queimadas: 75906
contencao: 66455
total_ignicoes: 1452
pico_ignicoes: 96 1419
percentual_queimado: 1.55
percentual_protegido: 1.33
checksum: 5220645806650296547
tempo: 26.238216
```

Utilizaremos deste formato de métricas para embasar a comparação quantitativa entre os resultados sequenciais e paralelizados que discutiremos nas seções seguintes.

## 4 Análise de códigos: comparativo entre sequencial e paralelo

Nesta seção descrevemos o formato geral do código estruturado para a solução sequencial e paralela do problema deste trabalho para o modelo de propagação de incêndios proposto. O código fonte na íntegra está disponível no link mencionado no início deste relatório. Neste relatório destacamos apenas os trechos críticos de performance.

O código pode ser dividido em 4 grandes etapas, além da leitura e validação de configurações de entrada (início de execução) e impressão de relatório final e liberação de memória (final de execução), de menor impacto computacional:

1. Construção de matriz linear e vetores de tempo;
2. Construção de mapa de contenção;
3. Aplicação de focos de incêndio;
4. Execução da simulação e cálculo de métricas por passo temporal;

A seguir detalhamos cada um dos itens acima.

### 4.1 Construção de matriz linear e vetores de tempo

Cada célula da matriz é definida pela struct **Célula**.

---

```
typedef struct {  
    int ID_ESTADO;  
    int ID_COBERTURA;  
    int FATOR_INCENDIO;  
    int UMIDADE;  
    int TEMPO_QUEIMA;  
} Celula;
```

---

Com matriz linear construída no formato **Celula\***, para melhor otimização quando consideramos Localidade Física.

---

```
Celula *matrix = malloc((size_t) configs->L * configs->C * sizeof(Celula));
```

---

Aqui vale ressaltar a importância da escolha de armazenamento linear ao invés de matricial (no formato **Celula\*\***). Quando o primeiro acesso é feito para a posição 0 da matriz linear, o bloco contíguo de dados é selecionado e armazenado nas memórias cache L1/L2. Dessa forma, todos os acessos subsequentes (1 ... N) apresentará **cache hit**, com melhoria expressiva na eficiência do código.

Além disso, ao armazenar os dados linearmente, as operações podem ser feitas de forma vetorial (modelo SIMD) de modo mais eficiente. Como uma dada linha  $l$  apresenta operação idêntica e utiliza de dados presentes na sua vizinhança próxima ( $l - 1$  e  $l + 1$ , com exceção aos casos de borda  $l = 0$  e  $l = L - 1$ ), operação vetorial pode ser empregada simultaneamente por cada núcleo de CPU.

## 4.2 Construção de mapa de contenção

De modo análogo podemos analisar a construção dos mapas de contenção, onde utilizamos a relação linear **int\*** para armazenar o passo  $p$  em que determinada célula de índice  $i$  terá sua zona de contenção ativada. Vale notar o uso de operações inteiras, em oposição a um potencial uso de divisão e resto de divisão. A função descrita está definida no código como **build\_mapa\_contencao**.

As operações de divisão e módulo são computacionalmente caras em relação a operações de soma e multiplicação, as quais também apresentam suporte de vetorização automática (quando utilizada flag *-O3* por exemplo)

## 4.3 Aplicação de focos de incêndio

Utilizamos estratégia similar na aplicação dos focos iniciais de incêndio e população inicial da matriz, com geração de cobertura, fator de incêndio, estado e umidade. Embora não faça parte de execução a ser paralelizada, é um importante recurso de otimização. As funções descritas estão definidas no código como **apply\_focos\_iniciais\_incendio**, responsável pela aplicação de focos iniciais de incêndio, e **populate\_matrix**, responsável pela população da matriz, com preenchimento de cada célula.

## 4.4 Execução da simulação e cálculo de métricas por passo temporal

Para a execução da simulação propriamente dita temos 5 operações principais

1. Ativação de zonas de contenção;
2. Atualização da matriz de próximo tempo;
3. Cálculo de métricas;
4. Troca de matrizes;
5. Verificação de condição de parada.

Desta forma podemos operar com a paralelização nos instantes desejados, a citar: ativação das zonas, atualização da matriz, redução para contadores, troca de matriz de forma segura e compartilhamento correto de condição

de parada. Destacamos algumas das decisões de implementação sequencial que serão importantes quando na aplicação paralela.

#### 4.4.1 Ativação de zonas de contenção

Para a ativação de zonas de contenção, precisamos percorrer todo o vetor de ativação (LxC) para acioná-las em células intactas no instante  $p$  correto. A função descrita está definida no código como **activate\_zonas\_contencao**

Neste formato, temos um único dado compartilhado (passo  $p$ ), acessível apenas como leitura, e a alteração em memória ocorre apenas para posições independentes (índice  $i$  do vetor **Celula\* matrix**), o que facilita a paralelização das operações, uma vez que não existem condições de exclusão mútua ou de corrida.

#### 4.4.2 Atualização da matriz de próximo tempo

Para a atualização da matriz, operamos a transição de estados, o cálculo do potencial de ignição por célula e a contabilização do tempo de queima. A função descrita está definida no código como **update\_matrix**

A atualização da matriz, assim como a ativação de zonas de contenção, deverá percorrer toda a matriz de estados para operar as transições necessárias. Dessa forma, manter operações inteiras (apenas multiplicação e soma) e percorrer linearmente os dados garante melhor otimização do código considerando a localidade espacial de dados para busca em memória.

Além disso, vale ressaltar que utilizamos como base deste trabalho os chamados autômatos celulares, que são tipicamente utilizados para descrever fenômenos em que seu estado atual apresenta dependência única e direta de seu estado anterior. Assim, ao utilizarmos duas matrizes (**matrix\_atual** e **matrix\_proximo**) permitimos que todo conteúdo de **matrix\_atual** seja operado apenas como leitura e que os índices acessíveis por cada iteração alterem dados apenas sobre **matrix\_proximo**, garantindo independência de dados para as futuras operações paralelas.

#### 4.4.3 Cálculo de métricas

Para calcularmos as métricas por instante  $p$ , precisamos contabilizar o número de células em um dado estado e o número de ignições (transição de estado  $1 \rightarrow 2$ , "intacta" para "em chamas"). Para facilitar o armazenamento, definimos uma struct **Metrics** no formato abaixo.

---

```
typedef struct {  
    int PASSO;  
    int COMBUSTIVEIS;  
    int NAO_COMBUSTIVEIS;
```

```
int INTACTAS;
int EM_CHAMAS;
int QUEIMADAS;
int CONTENCAO;
int TOTAL_IGNICOES;
double TEMPO;
} Metrics;
```

---

Assim, definimos um vetor **Metrics\*** de tamanho  $P$  (máximo de passos da simulação) capaz de armazenar, por instante  $p$ , o acumulado de cada métrica. Como precisaremos percorrer toda a matriz e acumular o resultado (e.g. somar todas as células em estado 0, i.e. "não combustível"), temos competição entre as variáveis acumuladas, tipicamente resolvido pelo formato **reduce**. A função descrita está definida no código como **calculate\_metrics\_resultados**

Deste modo, quando no momento da paralelização, conseguimos alocar cópias privadas de cada uma das variáveis em cada thread construída para que, ao final, seja possível sincronizar o resultado final de soma em estrutura de árvore, com redução por algoritmo de complexidade  $O(\log n)$ , melhorando assim a performance da execução.

#### 4.4.4 Troca de matrizes

Já nas ações finais da simulação, neste momento executamos a troca de ponteiros (swap) para que a matriz de próximo estado seja compreendida como a matriz atual (que será lida e utilizada para calcular os próximos estados) e para que a matriz atual seja compreendida como a matriz de próximo estado (que terá operações de escrita sobre suas células). Caso a operação de swap seja realizada enquanto outros threads estão operando, a leitura e escrita serão feitas sobre memórias indesejadas.

Assim, é necessário que todos os threads finalizem suas operações anteriores (atualização de matriz e cálculo de métricas) no passo  $p$  para que seja possível iniciar  $p + 1$ .

A princípio poderíamos optar pela troca de matrizes na forma a seguir.

---

```
Celula* matrix_tmp = matrix_estado_atual;
matrix_estado_atual = matrix_proximo_estado;
matrix_proximo_estado = matrix_tmp;
```

---

No entanto neste formato temos dois problemas principais: o trecho de código deverá estar dentro de uma unidade de execução única (apenas um thread deve executar, após aguardo de finalização das demais) e causará **cache miss** para a próxima leitura, pois a variável **matrix\_proximo\_estado** foi alterada e será assinalada como inválida. Por estes motivos, uma possível construção desta função que evita os problemas mencionados é dada a seguir. A função descrita está definida no código como **run\_simulation**

#### 4.4.5 Verificação de condição de parada

Como último passo, verificamos as condições de parada. Ao final de cada iteração incrementamos nosso instante ( $p + +$ ), mas, uma vez que utilizamos um loop do-while, nosso incremento de passo  $p$  deverá ser decrementado para verificação da condição de parada ( $p - 1$ , ao fim do laço, refere-se à iteração que aconteceu no passo corrente). Aqui todos os threads deverão compartilhar do mesmo valor de  $p$  e, portanto, esta região é crítica e deve ter sua execução processada por um único thread, garantindo sincronia dos demais.

### 4.5 Paralelização e análise de resultados

Nesta seção aplicamos as diretivas de OMP para paralelizar trechos do código considerando as regiões destacadas nas seções anteriores.

Como primeira estratégia, utilizaremos diferentes tipos de schedules, sem emprego de região persistente. Deste modo, temos três funções paralelizáveis, destacadas na tabela 4.

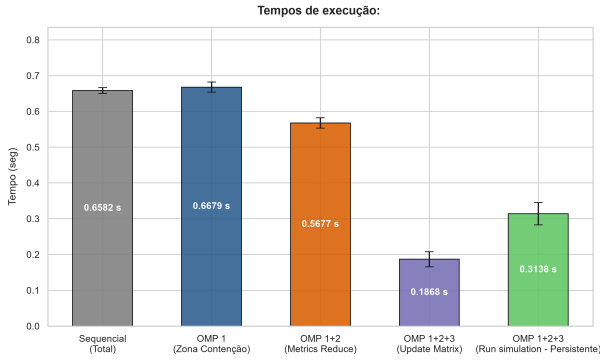
Para cada trecho, utilizamos os seguintes argumentos

- **parallel**: Cria um conjunto de threads para executar o bloco de código imediatamente em seguida. A execução é replicada para todos threads.
- **for**: Divide as iterações para o for-loop imediatamente próximo.
- **schedule(schType)**: Define como as iterações serão particionadas e distribuídas por cada thread, com balanceamento de carga de trabalho.
- **reduction(+: [vars,])**: Aloca múltiplas cópias das variáveis para cada thread inicializada e opera sobre cada variável para que, ao final, possam ser acumuladas em uma soma resultante.
- **collapse(2)**: Combina dois loops aninhados em uma única iteração (for i in L ... for j ... in C transforma-se em for x in  $L \times C$ ) para que seja distribuída pelas cargas de trabalho.

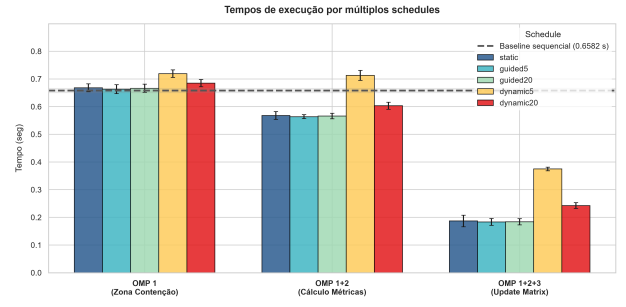
| Função                       | Diretiva                                                        |
|------------------------------|-----------------------------------------------------------------|
| activate_zonas_contencao     | #pragma omp parallel for schedule(static)                       |
| calculate_metrics_resultados | #pragma omp parallel for schedule(static) reduction(+: [vars,]) |
| update_matrix                | #pragma omp parallel for schedule(static) collapse(2)           |

Table 4: Relação entre função e diretiva de paralelização OMP

Com as cláusulas acima, analisamos agora as diferenças em tempo de processamento quando adicionamos as diretivas para cada função e para diferentes tipos de schedule. Os resultados podem ser observados na figura 6



(a) Tempos de execução para implementação (da esquerda para a direita) Sequencial, OMP1, OMP1 em conjunto com OMP2, OMP1 em conjunto com OMP2 e OMP3.



(b) Tempos de execução para implementação incremental (da esquerda para a direita) OMP1, OMP1 em conjunto com OMP2, OMP1 em conjunto com OMP2 e OMP3, considerando múltiplos schedules.

Figure 6: Tempos de execução para múltiplas implementações, considerando as regiões de Ativação de Zona de Contenção (OMP1), Cálculo de métricas (OMP2) e Atualização de matriz (OMP3). Os valores representam a média de 10 execuções independentes.

Abaixo apresentamos os arquivos de entrada e suas respectivas saídas para a solução paralela, mantendo schedule static, não persistente (melhor cenário). Vale ressaltar que na versão final utilizamos threads persistentes, diferentemente do que mostramos na tabela 4. Para evitar a sobrecarga de processamento para a criação e destruição de um número N de threads, podemos optar em criá-las para uma região com trechos paralelizáveis e, para cada bloco paralelo (for-loops por exemplo) apenas utilizar do conjunto de threads disponíveis. Neste contexto, as funções apresentam as seguintes diretivas na tabela 5 abaixo.

Na tabela 5, três argumentos são distintos dos já utilizados antes e relevantes, citamos:

- **default(**none**)**: define o escopo não padrão de variáveis por thread construída. Ao especificarmos **none**, cada variável visível para blocos paralelos deverão ser explicitamente definidas, evitando erros de execução;
- **shared([vars,])**: em conjunto ao argumento default(**none**), agora precisamos definir as variáveis que serão compartilhadas por threads. Particularmente útil neste caso para operar com **reduction**, onde operamos sobre um conjunto de variáveis e as somamos no final;
- **simd**: explicita ao compilador para estruturar o loop-for no modelo SIMD, de forma vetorial. Vale ressaltar aqui a importância de não existir dependências de dados de iterações anteriores. O modelo SIMD pressupõe independência entre as operações de cada iteração.

Por fim, detalhamos abaixo os resultados das operações sobre cada uma das entradas fornecidas para o código em modelo paralelo, importante para verificar a identidade do comportamento do código em relação à sua versão sequencial quanto às métricas calculadas e também com o comparativo do tempo de execução.

| Função                       | Diretiva                                                          |
|------------------------------|-------------------------------------------------------------------|
| run_simulation               | #pragma omp parallel num_threads(T) default(none) shared([vars,]) |
| activate_zonas_contencao     | #pragma omp for simd schedule(static)                             |
| calculate_metrics_resultados | #pragma omp for simd schedule(static) reduction(+: [vars,])       |
| update_matrix                | #pragma omp for simd schedule(static) collapse(2)                 |

Table 5: Relação entre função e diretiva de paralelização OMP considerando threads persistentes e cláusula default(none).

#### entrada\_carga\_pequena.txt

```
400 500 80 4 2027 35
0 1 3
3 2
100 83
200 250
300 416
15 70 150 130 165
35 245 320 355 335
```

#### output.out

```
passos: 80
nao_combustiveis: 39704
intactas: 138809
em_chamas: 376
queimadas: 18889
contencao: 2222
total_ignicoes: 372
pico_ignicoes: 66 398
percentual_queimado: 12.02
percentual_protegido: 1.39
checksum: 35988890671224108
tempo: 0.043794
```

#### entrada\_carga\_media.txt

```
1200 1500 100 8 3031 35
1 1 2
6 4
171 125
342 375
514 625
685 876
857 1125
1028 1375
20 250 450 450 470
40 500 780 780 800
60 760 1080 1030 1100
75 950 1280 1150 1300
```

#### output.out

```
passos: 100
nao_combustiveis: 360809
intactas: 1356445
em_chamas: 1191
queimadas: 65510
contencao: 16045
total_ignicoes: 1185
pico_ignicoes: 90 1228
percentual_queimado: 4.63
percentual_protegido: 1.11
checksum: 2450667959220978121
tempo: 0.490392
```

entrada\_carga\_grande.txt

```
2500 2500 100 8 4099 35
-1 1 4
10 6
227 125
454 375
681 626
909 875
1136 1125
1363 1375
1590 1626
1818 1875
2045 2125
2272 2375
15 150 300 500 325
30 550 700 900 730
45 950 1100 1350 1135
60 1350 1500 1750 1540
75 1750 1900 2150 1945
90 2050 2200 2400 2250
```

output.out

```
passos: 100
nao_combustiveis: 1251181
intactas: 4855039
em_chamas: 1419
queimadas: 75906
contencao: 66455
total_ignicoes: 1452
pico_ignicoes: 96 1419
percentual_queimado: 1.55
percentual_protegido: 1.33
checksum: 5220645806650296547
tempo: 1.944326
```

Na tabela 6 apresentamos uma relação entre os tempos de execução para cada uma das entradas. Aqui as métricas são referentes a uma única execução, comparando apenas os resultados de saída apresentados anteriormente.

| entrada                   | $T_{seq}$ Tempo sequencial | $T_{omp}$ Tempo paralelo | $T_{seq}/T_{omp}$ |
|---------------------------|----------------------------|--------------------------|-------------------|
| entrada_carga_pequena.txt | 0.648286                   | 0.043794                 | 14.803078         |
| entrada_carga_media.txt   | 10.998731                  | 0.490392                 | 22.428447         |
| entrada_carga_grande.txt  | 26.238216                  | 1.944326                 | 13.494761         |

Table 6: Compartivo entre tempos de execução para versões sequencial e paralela para cada entrada fornecida.

Por fim, quando analisamos no gráfico 7 para a métrica de *speedup* em relação ao número de threads utilizadas. Em relação à métrica linear (ou seja, se para N threads notamos uma melhoria em N vezes no desempenho do código) fica evidente a relação sublinear da otimização proposta. Embora não ideal, é um cenário recorrente para algoritmos paralelos proposto, com a manutenção de uma região de estabilidade, onde o aumento do número de threads não afeta o desempenho frente ao tempo de execução.

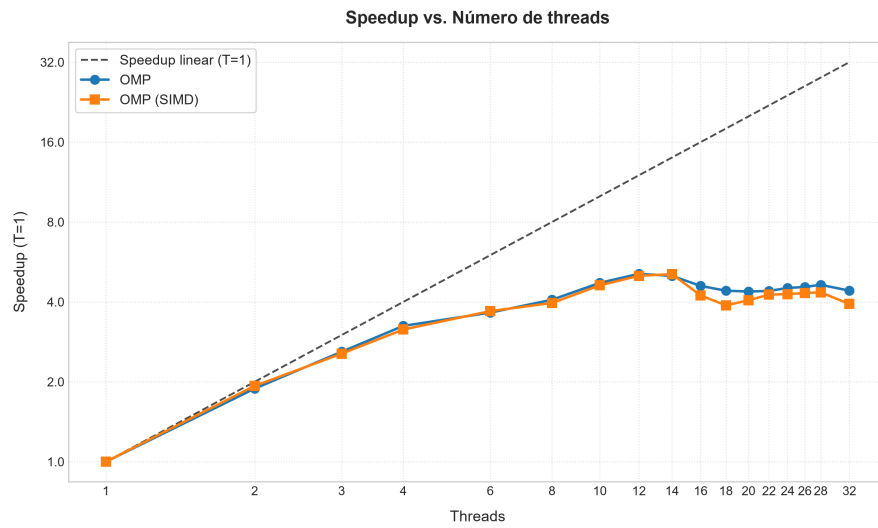


Figure 7: Speedup considerando o arquivo de entrada entrada\_carga\_media.txt. Notamos um comportamento sublinear para o código paralelizado. Quando utilizamos junto à diretiva pragma omp o argumento **simd** para vetorização. Cada ponto de dado representa a média de 10 execuções independentes.